

OpenCV-Based Computer Vision System Designs

Contents

1. Real-Time Face Recognition for Campus Security
2. Intelligent Traffic Violation Detection System
3. Automated Document Processing System
4. Real-Time Multi-Person Tracking and Attendance System
5. Smart Home Surveillance and Intrusion Detection

1. Real-Time Face Recognition for Campus Security

The goal is to build an OpenCV-based pipeline that captures faces from live CCTV streams, detects them reliably under varying lighting, extracts discriminative features, recognizes registered students and staff, and does all of this in real time on a low-cost computer while keeping false recognitions low. The design below breaks the system into six stages that mirror the actual data flow through the pipeline.

Fig 1.1 - Real-Time Face Recognition Pipeline (Campus Security)

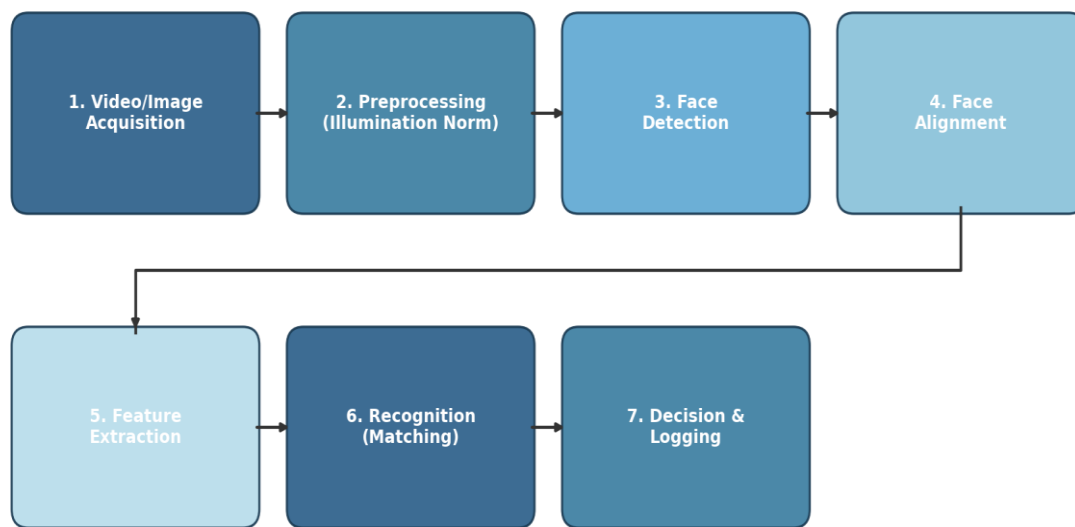


Figure 1.1: End-to-end pipeline for real-time campus face recognition.

1.1 Image Acquisition

Video frames are pulled from the CCTV/IP camera using `cv2.VideoCapture` (RTSP/ONVIF stream or USB webcam). To sustain real-time throughput on modest hardware, frames are read in a dedicated capture thread that continuously fills a small ring buffer, while a separate processing thread consumes the latest frame. This decouples slow processing from frame arrival and prevents queue backlog. Frame resolution is downscaled (e.g., 1280x720 to 640x360) before detection since face detection cost scales with pixel count, and the full-resolution frame is retained only for the final crop used in feature extraction, preserving detail where it matters.

1.2 Preprocessing

Because lighting at an entrance changes with time of day, weather, and backlighting, preprocessing focuses on normalizing illumination and reducing noise before detection:

- **Grayscale conversion** (`cv2.cvtColor`, `COLOR_BGR2GRAY`) for detectors that operate on intensity, reducing computation versus 3-channel processing.
- **Histogram equalization / CLAHE** (`cv2.createCLAHE`) applied locally in tiles to boost contrast in shadowed or overexposed regions without over-amplifying noise globally, which plain histogram equalization tends to do.
- **Gaussian blur** (`cv2.GaussianBlur`, small 3x3 kernel) to suppress sensor noise before edge-sensitive detection stages.
- **Gamma correction** for very dark frames captured at night or under IR-only illumination.

1.3 Face Detection

A cascade of two detector options is used depending on available compute:

- **Fast tier:** OpenCV's DNN face detector (a lightweight SSD/ResNet-10 model shipped with OpenCV, loaded via `cv2.dnn.readNetFromCaffe`) run at low input resolution (300x300). This is substantially more robust to pose and lighting than Haar cascades while remaining CPU-friendly.
- **Fallback tier:** Haar cascade (`cv2.CascadeClassifier`, `haarcascade_frontalface_default`) as a cheap secondary check or on extremely constrained hardware, accepting a lower recall in exchange for speed.

Detected face boxes are passed through non-maximum suppression to eliminate duplicate boxes, and a minimum-size filter discards faces too small to yield reliable features (e.g., under 60x60 px), which also filters out background pedestrians far from the entrance.

1.4 Face Alignment

Before feature extraction, each detected face is aligned using eye-landmark positions (via OpenCV's facemark LBF model or a lightweight landmark predictor). The face is rotated and scaled so the eye line is horizontal and inter-eye distance is normalized. Alignment significantly reduces intra-class variation caused by head tilt, which otherwise degrades matching accuracy.

1.5 Feature Extraction

Two approaches are combined depending on hardware budget:

- **Deep embeddings (preferred):** A pre-trained lightweight embedding network (e.g., OpenFace or a MobileFaceNet-style model run through `cv2.dnn`) converts each aligned face into a 128-256 dimension embedding vector. Embeddings from the same identity cluster tightly in this space regardless of moderate lighting or pose changes, giving far better accuracy than handcrafted features.
- **Classical fallback (LBPH):** `cv2.face.LBPHFaceRecognizer_create` extracts Local Binary Pattern Histograms, which are cheap to compute and reasonably illumination-tolerant, useful when no GPU/DNN acceleration is available at all.

1.6 Recognition (Matching)

For deep embeddings, recognition is a nearest-neighbor search against a pre-enrolled gallery of embeddings (one or more samples per registered student/staff, captured during enrollment). Cosine similarity or Euclidean distance is computed, and the top match is accepted only if the similarity clears a calibrated threshold; otherwise the person is flagged “unknown.” For LBPH, `cv2`'s built-in `predict()` method returns a confidence/distance score that is thresholded the same way. Using a distance threshold rather than always returning the closest match is the key control that keeps the false-accept rate low.

1.7 Optimization for Real-Time Performance on Low-Cost Hardware

Technique	Effect
Frame skipping / detect every Nth frame	Detection runs every 2-3 frames; tracking (e.g., <code>cv2</code> KCF/CSRT tracker) carries the box forward in between, cutting detector calls significantly.
Downscaled detection input	Detector runs on a resized frame; the resulting box is scaled back to full resolution for cropping, reducing detector cost roughly quadratically with the scale factor.
Multithreading	Capture, detection, and recognition run in separate threads/queues so no stage blocks the others.
Model quantization	DNN models are used in INT8/FP16 where supported by OpenCV's DNN backend to speed inference on CPU.

Technique	Effect
Region-of-interest limiting	Detection is restricted to the entrance zone in the frame (the area people must walk through), ignoring irrelevant background.

1.8 Why This Achieves the Required Accuracy and Speed

Accuracy is achieved through the combination of illumination normalization (CLAHE/gamma), a DNN detector that is robust to pose/lighting, geometric alignment before matching, and deep embeddings with a calibrated distance threshold rather than forced top-1 matching, which directly minimizes false recognitions. Real-time performance is achieved by downscaling detection input, only running the expensive detector periodically while a cheap tracker interpolates between detections, and pipelining capture/detection/recognition across threads so total latency is bounded by the slowest stage rather than the sum of all stages.

2. Intelligent Traffic Violation Detection System

This system monitors a roadside camera feed to detect, track, and count vehicles crossing a virtual line, while flagging abnormal movement (e.g., wrong-direction travel, sudden lane changes, stopping in a no-stop zone).

Fig 2.1 - Traffic Violation Detection Pipeline

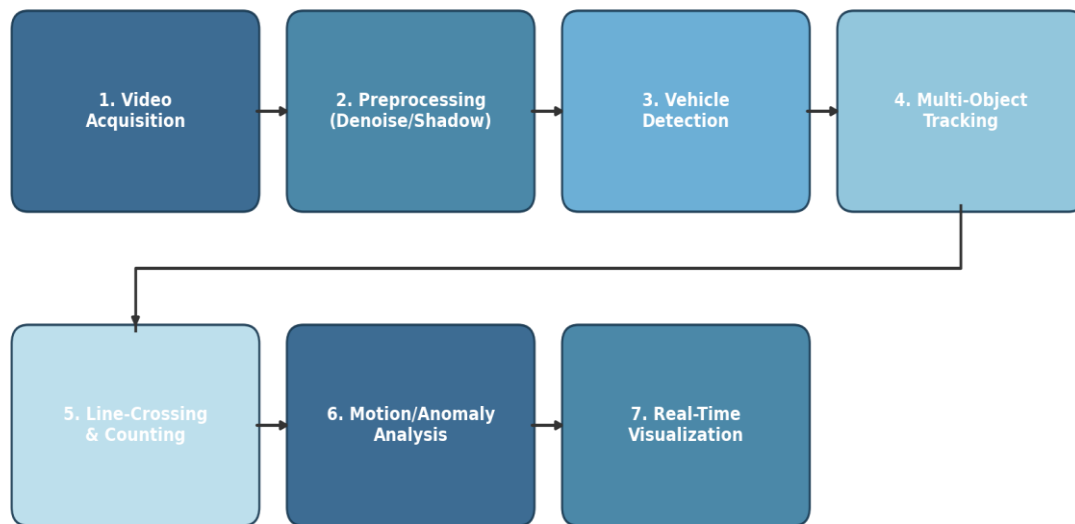


Figure 2.1: Pipeline for vehicle detection, tracking, counting, and violation alerts.

2.1 Video Acquisition and Preprocessing

Frames are captured via `cv2.VideoCapture` from the roadside camera. Preprocessing steps address common outdoor artifacts:

- **Gaussian/Median blur** to reduce sensor and compression noise before background modeling.
- **Frame resizing** to a fixed working resolution for consistent processing speed regardless of source resolution.
- **ROI masking** to restrict analysis to the road surface and exclude sidewalks, sky, or moving trees at the frame edges.

2.2 Vehicle Detection

Two complementary approaches are used:

- **Background subtraction (motion-based):** `cv2.createBackgroundSubtractorMOG2` (with shadow detection enabled) or KNN builds an adaptive background model; foreground blobs above a minimum contour area are candidate vehicles. This is lightweight and works well for consistent traffic flow.
- **DNN object detector (appearance-based):** A lightweight model such as MobileNet-SSD or YOLO-tiny, run through `cv2.dnn`, detects and classifies vehicles (car/bus/truck/motorbike) per frame independent of motion, which catches stationary or slow-moving vehicles that background subtraction can miss.

Combining both: background subtraction proposes fast candidate regions, and the DNN detector confirms/classifies them, which reduces false positives from waving trees or lighting flicker while keeping computation manageable.

2.3 Tracking Across Frames

Each detected vehicle is assigned a tracker instance. A centroid-tracking algorithm (matching detections frame-to-frame by nearest centroid distance and bounding-box IoU, similar to a lightweight SORT) assigns and maintains a persistent ID for each vehicle. For short occlusions (e.g., one vehicle briefly hidden behind another), a Kalman filter predicts the expected position so the ID is retained rather than a new one being created when the vehicle reappears.

2.4 Line-Crossing Detection and Counting

A virtual counting line is defined as a fixed pixel coordinate segment. Each tracked vehicle's centroid trajectory is checked frame-to-frame: when the centroid transitions from one side of the line to the other, the count for that direction is incremented exactly once, keyed to that vehicle's unique tracker ID. Using the ID (not just “a blob crossed”) is what prevents the same vehicle from being counted multiple times if it jitters near the line.

2.5 Motion Analysis for Abnormal Behavior

Using each vehicle's tracked trajectory, the system computes velocity direction and magnitude between frames:

- **Wrong-way detection:** flags a vehicle whose direction vector opposes the expected lane direction beyond a tolerance angle.

- **Sudden stop / illegal parking:** flags a vehicle whose centroid stays within a small radius for longer than a set duration inside a no-stopping zone.
- **Erratic lane change:** flags large lateral displacement per frame relative to the vehicle's own recent trajectory baseline.

2.6 Visualization

Bounding boxes, tracker IDs, the counting line, and running counts (per direction, per vehicle class) are drawn on the frame using `cv2.rectangle/putText`, and alerts are overlaid in a distinct color with a text banner when a violation is flagged. The annotated stream is shown via `cv2.imshow` or pushed to a dashboard.

2.7 Handling Shadows, Noise, Dynamic Backgrounds, and Multiple Vehicles

Challenge	Mitigation
Shadows	MOG2's built-in shadow detector marks shadow pixels with a distinct gray value (127) which is explicitly excluded from the foreground mask before contour extraction; morphological opening additionally strips thin shadow slivers.
Camera noise	Gaussian/median blur pre-filtering plus a minimum-contour-area threshold discards single-pixel or small speckle detections.
Dynamic backgrounds (trees, flags)	ROI masking excludes non-road regions; MOG2's adaptive learning rate absorbs slow repetitive motion into the background model over time so it stops triggering detections.
Multiple/overlapping vehicles	DNN detection with class-aware bounding boxes plus IoU-based tracker association separates overlapping vehicles better than blob-only contour analysis, and the Kalman filter bridges brief occlusion without merging or losing IDs.

3. Automated Document Processing System

This pipeline takes noisy scanned documents (blur, uneven lighting, skew, varying font sizes) and prepares them for reliable OCR, processed in batch across thousands of files.

Fig 3.1 - Document Preprocessing & OCR Pipeline

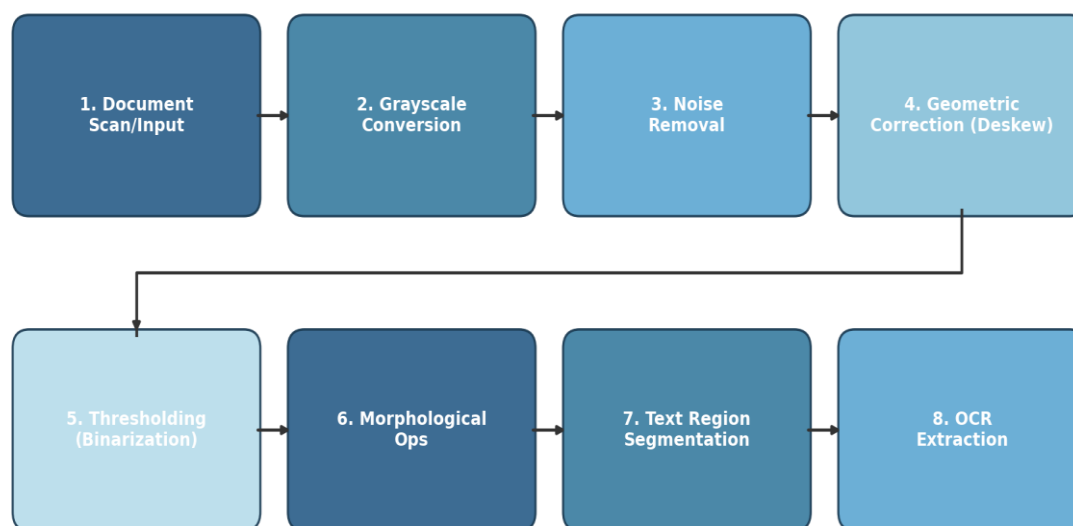


Figure 3.1: Document preprocessing pipeline feeding into OCR.

3.1 Grayscale Conversion

Every scanned page is converted with `cv2.cvtColor(img, COLOR_BGR2GRAY)`. OCR engines operate on intensity/structure, not color, so this immediately removes irrelevant chrominance information, reduces memory and computation for every following step, and is a prerequisite for thresholding.

3.2 Noise Removal and Background Cleanup

- **Median blur** (`cv2.medianBlur`) removes salt-and-pepper speckle noise common in scans while preserving character edges better than Gaussian blur.
- **Non-local means denoising** (`cv2.fastNlMeansDenoising`) is used for heavier noise (e.g., low-quality scans) since it removes noise while keeping fine text strokes intact.
- **Background/illumination correction:** uneven lighting is corrected by estimating the background via a large-kernel morphological closing or Gaussian blur, then dividing the

original image by this background estimate, which flattens illumination gradients across the page before thresholding.

3.3 Geometric Correction (Deskew)

Scanned pages are often slightly rotated. The skew angle is estimated either via the minimum-area bounding rectangle of text-pixel contours (`cv2.minAreaRect`) or via Hough line detection on text baselines, and the page is rotated back with `cv2.warpAffine`. Correct orientation matters because most OCR engines assume horizontal text lines; even a few degrees of skew measurably drops recognition accuracy.

3.4 Thresholding (Binarization)

Adaptive thresholding (`cv2.adaptiveThreshold`, Gaussian-weighted) is preferred over a single global threshold because illumination can still vary locally across a page even after background correction. Otsu's method (`cv2.threshold` with `THRESH_OTSU`) is used as a faster alternative for uniformly lit documents. Binarization converts the page into clean black text on a white background, the format OCR engines are tuned for.

3.5 Morphological Operations

- **Opening** (erosion followed by dilation) removes small noise specks left after thresholding.
- **Closing** (dilation followed by erosion) reconnects character strokes broken by scan artifacts or thin fonts.
- **Dilation with a wide horizontal kernel** merges individual characters into word/line blobs, which is used specifically to help the next segmentation stage locate text lines.

3.6 Text Region Segmentation

Contours are extracted (`cv2.findContours`) from the dilated text-line blobs to obtain bounding boxes for paragraphs, lines, or columns depending on the dilation kernel used. Boxes are sorted top-to-bottom, left-to-right to preserve reading order, and very small or very large (non-text) contours are filtered out using aspect-ratio and area heuristics, which also helps separate embedded tables/images from body text before OCR.

3.7 Text Extraction and Batch Processing

Each segmented region is cropped and passed to Tesseract OCR (via pytesseract) individually rather than the whole page, which improves accuracy since the OCR engine works on a smaller, cleaner region with consistent font size. For batch mode, documents are processed through a multiprocessing pool (one worker per CPU core), with each document's stages 3.1-3.7 run independently and results written incrementally to a database/output folder, so a single corrupt file does not stall the batch.

3.8 How Each Stage Improves OCR Accuracy

Stage	OCR Accuracy Contribution
Grayscale conversion	Removes irrelevant color noise, standardizes input for thresholding.
Illumination correction	Prevents shadowed regions from being mis-thresholded as solid black.
Denoising	Stops OCR from mis-reading noise speckles as spurious characters (e.g., periods, commas).
Deskew	Aligns text horizontally, matching OCR's line-segmentation assumptions.
Adaptive thresholding	Produces crisp, consistent black-on-white text regardless of local lighting.
Morphological cleanup	Repairs broken strokes and removes residual noise blobs before recognition.
Segmentation	Lets OCR focus per-region, improving accuracy on mixed font sizes and multi-column layouts.

4. Real-Time Multi-Person Tracking and Attendance System

This system uses classroom CCTV to detect all faces in view, track each person across frames, recognize registered students, and mark attendance exactly once per class session, while tolerating brief head turns or occlusion.

Fig 4.1 - Multi-Person Tracking & Attendance Pipeline

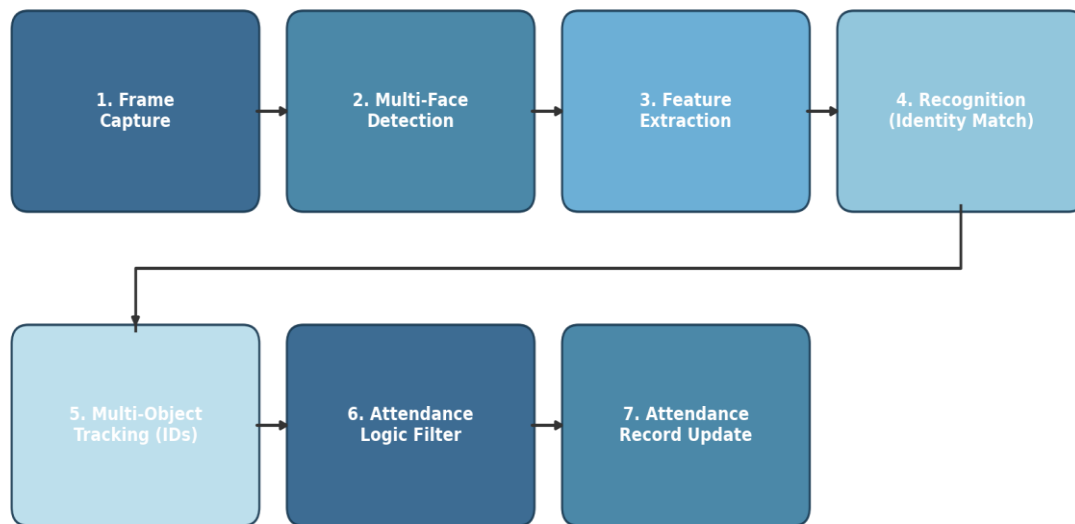


Figure 4.1: Multi-person detection, tracking, and attendance-logging pipeline.

4.1 Multi-Face Detection

Each frame is passed through the same DNN face detector described in Question 1 (cv2.dnn with a lightweight SSD-based face model), which natively returns all faces above a confidence threshold in one forward pass rather than needing a separate call per person, making it efficient for a classroom with many students in frame simultaneously.

4.2 Feature Extraction and Recognition

Each detected face is aligned and passed through the same embedding network used in Question 1 to produce an identity vector, then matched against the enrolled student gallery by nearest-neighbor distance, accepted only above a calibrated similarity threshold to avoid mis-marking attendance for a similar-looking but different student.

4.3 Multi-Object Tracking to Maintain Identity Across Frames

Rather than re-running recognition on every frame for every face (expensive with many students), each recognized face is handed to a lightweight per-person tracker (cv2 tracker such as CSRT/KCF, or a centroid+IoU tracker as in Question 2) that follows the face box across subsequent frames using only cheap visual tracking, not full recognition. Each tracker carries the identity label assigned at the last successful recognition. Recognition is then only re-run periodically (e.g., every 1-2 seconds) or when a tracker is lost, which keeps per-frame cost low while still confirming identity regularly enough to catch a mismatch.

4.4 Handling Head Movement and Temporary Disappearance

- **Short-term occlusion:** when a tracked face temporarily fails detection (student looks down, is blocked by another student), the tracker is kept alive for a grace window (e.g., a few seconds) using its last known position and a Kalman-filter motion prediction, rather than immediately dropping the identity.
- **Re-identification on reappearance:** if the tracker is lost beyond the grace window, the next detection at a nearby position is re-matched against the stored embedding of the most recently lost track before being treated as a brand-new face, which prevents creating a duplicate track for the same student.

4.5 Preventing Duplicate or False Attendance Records

Attendance logic is decoupled from per-frame recognition: a student's attendance is marked only once their tracked identity has been confirmed with high confidence for a minimum consecutive duration (e.g., stable recognition across several seconds), which filters out one-off misrecognitions. Once marked present for the session, that student's ID is added to a session-scoped “already marked” set, so any later re-detection of the same identity (e.g., after leaving and re-entering frame) is checked against this set and skipped, guaranteeing a single attendance record per student per session regardless of how many times they are re-detected.

4.6 Continuous, Low-Delay Operation

The same optimizations from Question 1 apply: threaded capture/processing, detection on downscaled frames, periodic (not per-frame) full recognition, and per-track lightweight visual

tracking in between. Because tracking is far cheaper than detection+recognition, overall processing delay stays low even as the number of simultaneously visible students grows.

5. Smart Home Surveillance and Intrusion Detection

This system watches a fixed entrance camera continuously, distinguishes real intrusion from noise/shadows/background clutter, tracks moving objects, and raises an alert only when something enters a defined restricted zone.

Fig 5.1 - Smart Home Intrusion Detection Pipeline

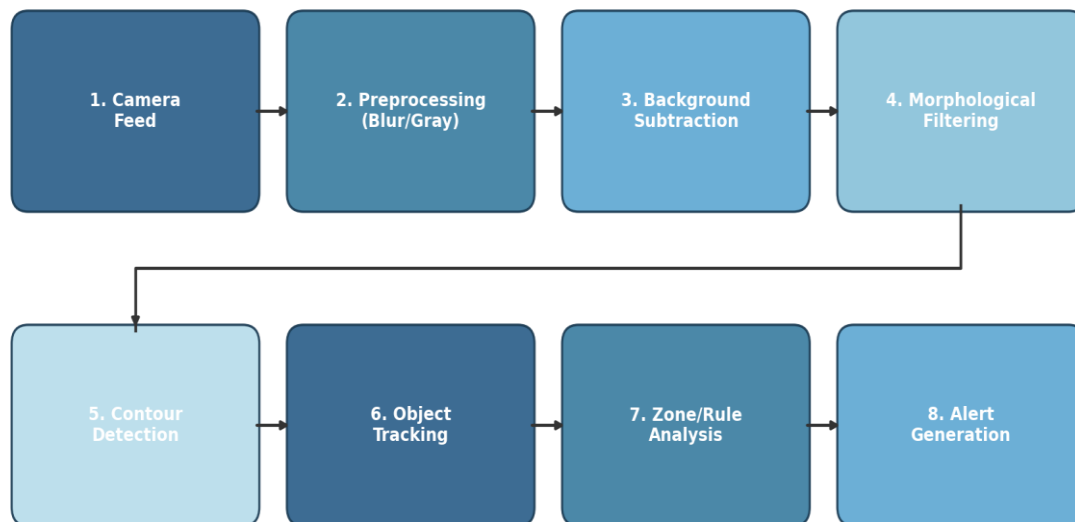


Figure 5.1: Background-subtraction based intrusion detection pipeline.

5.1 Preprocessing

Frames are converted to grayscale for the motion model and lightly blurred (`cv2.GaussianBlur`) to suppress per-pixel sensor noise that would otherwise register as false motion. Since the camera is fixed, no perspective correction is needed, simplifying the pipeline relative to Questions 2 and 3.

5.2 Background Subtraction

`cv2.createBackgroundSubtractorMOG2` (or `KNN`) builds and continuously updates a statistical model of the static scene. Each new frame is compared against this model to produce a foreground mask of pixels that differ significantly from the learned background. A moderate learning rate is used so that the background model slowly adapts to permanent changes (e.g., a parked car staying in one place) without absorbing a genuine intruder who is still moving.

5.3 Filtering and Morphological Operations

- **Shadow suppression:** MOG2's shadow flag pixels are excluded from the foreground mask, removing the elongated dark region a person's shadow creates, which would otherwise be misread as part of (or a separate) moving object.
- **Opening** removes small noise blobs from lighting flicker or leaf motion.
- **Closing** fills small holes inside a real moving object's silhouette so it forms one solid contour rather than fragments.

5.4 Contour Detection and Object Tracking

`cv2.findContours` extracts blobs from the cleaned foreground mask; a minimum-area threshold discards residual small-noise contours (leaves, insects, light flicker), keeping only blobs consistent with a person-sized object. Surviving contours are tracked frame-to-frame with a centroid/Kalman tracker (as in Questions 2 and 4) so each object keeps a persistent identity and trajectory rather than being treated as a new detection every frame.

5.5 Motion Analysis and Suspicious-Activity Identification

Using each tracked object's trajectory, duration, and size, the system classifies behavior: sustained directional movement toward the entrance is treated as approach; loitering (low displacement over an extended time) near the entrance can be flagged separately; and a size/aspect-ratio filter helps distinguish a person-shaped blob from a vehicle or animal-shaped one.

5.6 Zone-Based Alerting

A restricted polygon region (e.g., the doorway area) is defined once during setup. An alert triggers only when a tracked object's centroid enters this polygon (point-in-polygon test) and the object has been consistently tracked for a minimum number of frames, which avoids alerting on a single noisy frame. Alerts are logged with a timestamp and snapshot, and shown as an on-screen banner via `cv2.putText` plus an optional notification hook.

5.7 Reducing False Alarms

Source of False Alarm	Reduction Technique
Shadows	MOG2 shadow detection flag excluded from the foreground mask before contour extraction.
Illumination changes (clouds, auto-exposure, lights turning on)	Adaptive background learning rate absorbs gradual global brightness shifts; sudden large-scale mask changes across the whole frame are detected separately and treated as a lighting event (background reset) rather than as intrusion.
Trees / dynamic background	ROI masking excludes swaying vegetation from the analyzed region where possible; minimum contour area and aspect-ratio filtering remove small, irregular blobs typical of foliage motion.
Small animals, insects on lens	Minimum blob size and person-like aspect-ratio filtering exclude objects far smaller than a human silhouette.
Single-frame noise spikes	An object must be tracked consistently across several consecutive frames, and must actually enter the defined zone, before an alert is raised.

5.8 Operating Within Limited Compute

Background subtraction and contour analysis are computationally cheap compared to running a deep detector every frame, which is why this design leans on MOG2 as the primary detector for a fixed-camera scenario (unlike Questions 1, 2, and 4, where scene complexity justified a DNN). Frame downscaling, a bounded processing rate (e.g., 10-15 fps, sufficient for security monitoring), and restricting analysis to the relevant ROI keep the system usable on low-cost embedded or single-board hardware.